

Flow over surface-mounted ribs

Apurva Ambike

Department of Mechanical Engineering, Indian Institute of Technology Guwahati

Supervisor: Prof. Arnab Kr. De

1 Introduction

The flow over a bluff body is one of the most extensively studied problems in fluid mechanics. It is built on an intricate theoretical framework and comes with a wide range of applications. The character of the entire flow field around the body is largely dependent on the Reynolds number as flow transitions are dictated by it. The range of Reynolds number corresponding to these transitions depends on the geometrical properties of the body or system and the physical properties of the fluid. At low Reynolds number values, we see the formation of small recirculation bubbles. As the Reynolds number is increased, these recirculation bubbles grow in size and become unsteady. Beyond a certain value of Reynolds number, the unstable layers give rise to unsteady periodic vortex shedding, forming a vortex street. At still higher values of Reynolds number, the flow past a bluff body is fully turbulent. Flows in this regime are characterised by a broad range of interacting length and time scales. The analysis of such a flow requires complex mathematical treatment and extensive computational techniques. Their nature is strongly inhomogeneous and anisotropic, accompanied by non-local and non-linear interactions between flow mechanisms.

While the broader problem of turbulent bluff body flow involves this sophisticated hierarchy of interconnected phenomena, this study focuses on a deliberately simplified form of the bluff body flow in order to emphasise the underlying numerical and computational methodology. Specifically, we consider the two-dimensional incompressible laminar flow over two square ribs mounted on the bottom surface of a rectangular channel. We choose an optimum Reynolds number to obtain a steady-state solution, ensuring that the flow remains in the regime where the flow structures are stable, rather than exhibiting unsteady vortex shedding. Despite its relative simplicity, it is expected that we shall observe certain characteristic phenomena related to flow separation near sharp corners, formation of recirculation zones and a downstream wake. To solve this problem, the governing equations are cast in the vorticity-stream function formulation. This formulation reduces the problem to a coupled pair of partial differential equations, an elliptic equation for the stream function and a parabolic transport equation for vorticity. The equations obtained are discretized over a non-uniform grid using the Finite Difference Method.

A system of linear equations is obtained from the adopted discretization scheme. Methods to solve large, sparse systems of linear equations using iterative solution techniques such as Gauss-Seidel, Successive Over-Relaxation (SOR) and the Biconjugate Gradient Stabilized (BiCGSTAB) technique are examined along with the use of preconditioners to accelerate convergence. The performance of these techniques is comparatively studied in terms of their convergence rate. Finally, the flow field is obtained for the two cases: irrotational flow and viscous flow. The irrotational case is used as baseline to verify the numerical framework. The characteristics of both the cases are studied with particular emphasis on the separation and recirculation structures found in the full viscous case.

2 Physical system and mathematical model

2.1 Governing Equations

The fundamental equations governing any flow are the continuity equation and the Navier-Stokes equation:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \quad (2.1)$$

$$\frac{\partial \mathbf{V}}{\partial t} + (\mathbf{V} \cdot \nabla) \mathbf{V} = -\frac{1}{\rho} \nabla p + \mathbf{g} + \nu \nabla^2 \mathbf{V} \quad (2.2)$$

Two common simplifications are inviscid flow and irrotational flow. An inviscid flow is simply a flow where shear stresses are absent between fluid layers. An irrotational flow is characterised by a zero value of the curl of the vector field. In an irrotational flow, the fluid particles do not rotate about their axis. An inviscid flow can be rotational if there already exists a non-zero vorticity, such as in the case of a forced vortex. However, a viscous flow will produce rotationality in almost all cases as shear stresses will create deformation and rotation.

When we assume a steady, incompressible and irrotational flow without body forces such as gravity, the equations reduce to:

$$\nabla \cdot \mathbf{V} = 0 \quad (2.3)$$

$$(\mathbf{V} \cdot \nabla) \mathbf{V} = -\frac{1}{\rho} \nabla p \quad (2.4)$$

Moreover, in such a flow, we have the vorticity satisfying the equation:

$$\boldsymbol{\omega} = \nabla \times \mathbf{V} \implies \boldsymbol{\omega} = 0 \quad (2.5)$$

For an irrotational flow, we can introduce a companion function, the *potential function* ϕ , defined by:

$$\mathbf{V} = \nabla \phi \quad (2.6)$$

This guarantees that any continuous scalar function ϕ automatically satisfies the irrotationality condition because of a fundamental identity ($\nabla \times \nabla(\quad) \equiv 0$). Thus, in a two-dimensional Cartesian co-ordinate system, we can write the components of the vector field as:

$$u = \frac{\partial \phi}{\partial x}, \quad v = \frac{\partial \phi}{\partial y} \quad (2.7)$$

In a two-dimensional flow, we can also define the *stream function* ψ given as:

$$\mathbf{V} = \nabla \psi \times \hat{\mathbf{e}}_z \quad (2.8)$$

which results in:

$$u = \frac{\partial \psi}{\partial y}, \quad v = -\frac{\partial \psi}{\partial x} \quad (2.9)$$

This ensures that Eq. (2.3) is satisfied for any continuous ψ . Substituting u and v from Eq. (2.9) into the irrotationality condition Eq. (2.5), we obtain:

$$\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} = 0 \implies \nabla^2 \psi = 0 \quad (2.10)$$

Similarly, substituting u and v from Eq. (2.7) into the continuity equation Eq. (2.3), we obtain:

$$\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} = 0 \implies \nabla^2 \phi = 0 \quad (2.11)$$

Equations (2.10) and (2.11) are forms of Laplace's equation - a familiar equation that arises in science and engineering. Any function ψ or ϕ that satisfies Laplace's equation represents a possible two-dimensional, incompressible, irrotational flow field.

For the case of non-zero $\boldsymbol{\omega}$, we cannot define a potential function. However as the flow is still incompressible, the stream function can still be utilised. Writing the Navier-Stokes equation in the non-dimensional form,

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u} \quad (2.12)$$

Taking the curl of both the sides of the equation and substituting

$$\nabla \times \frac{\partial \mathbf{u}}{\partial t} = \frac{\partial \boldsymbol{\omega}}{\partial t}, \quad \nabla \times \nabla p = 0, \quad \nabla \times (\nabla^2 \mathbf{u}) = \nabla^2 \boldsymbol{\omega}, \quad \nabla \times (\mathbf{u} \cdot \nabla) \mathbf{u} = (\mathbf{u} \cdot \nabla) \boldsymbol{\omega} - (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} \quad (2.13)$$

we get the vorticity transport equation:

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + (\mathbf{u} \cdot \nabla) \boldsymbol{\omega} = (\boldsymbol{\omega} \cdot \nabla) \mathbf{u} + \frac{1}{Re} \nabla^2 \boldsymbol{\omega} \quad (2.14)$$

In two dimensions, the equation reduces to:

$$\frac{\partial \omega}{\partial t} + u \frac{\partial \omega}{\partial x} + v \frac{\partial \omega}{\partial y} = \frac{1}{Re} \left(\frac{\partial^2 \omega}{\partial x^2} + \frac{\partial^2 \omega}{\partial y^2} \right) \quad (2.15)$$

Moreover, the vorticity is

$$\omega = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y} \quad (2.16)$$

Substituting u and v from Eq. (2.9) into the irrotationality condition Eq. (2.16), we obtain:

$$\nabla^2 \psi = -\omega \quad (2.17)$$

This is Poisson's equation for the stream function. As u and v are dependent on ψ by Eq. (2.9) and they also appear in Eq. (2.15), equations (2.15) and (2.17) form a coupled system. The Laplace and Poisson equations are elliptic in nature while the vorticity transport equation (2.15) is parabolic. The values of ψ depend on ω instantaneously while ω evolves in time depending on ψ . The vorticity-stream function formulation offers a few advantages over using the primitive variable (velocity-pressure) formulation of Navier-Stokes equations. The pressure field is eliminated entirely from equations (2.15) and (2.17), removing the need for a pressure-velocity coupling scheme. Secondly, the continuity equation is satisfied by construction once u and v are defined using the stream function using Eq. (2.9), rather than being enforced as a separate constraint.

The drawback of this formulation is that the stream function is a two-dimensional construct and the vorticity in three dimensions is a vector governed by Eq. (2.14) which is more complex than its two-dimensional counterpart. For this reason, the primitive approach remains the standard choice for three-dimensional problems while the vorticity-stream function formulation continues to be used in two-dimensional problems, such as the one considered in this work, because of its simplicity and reduced computational costs.

2.2 Computational grid and Finite Difference Method

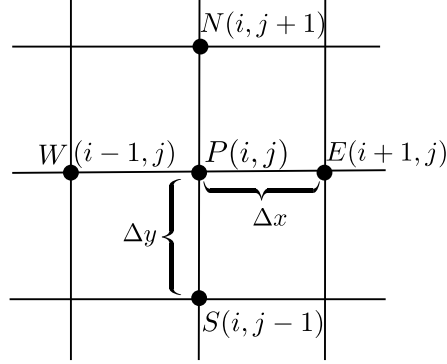


Figure 1: Notation for the finite difference stencil showing a point P and its four neighbouring nodes N , S , E and W on the grid.

Discretization is the process by which a closed-form mathematical expression, such as a function or a differential equation involving functions, all of which are viewed as having an infinite continuum of values throughout some domain, is approximated by analogous (but different) expressions which prescribe values at only a finite number of discrete points or volumes in the domain. Analytical solutions of partial differential equations involve closed-form expressions which give the variation of the dependent variables *continuously* throughout the domain. In contrast, numerical solutions give answers at *discrete points* in the domain, called *grid points*.

In Fig. 1, the grid points are identified by an index i which runs in the X direction and an index j which runs in the Y direction. Hence, if (i, j) are the indices for point P , then the point immediately to the east of P is E with indices $(i+1, j)$, the point to the west is W with indices $(i-1, j)$. Similarly, N and S denote the points to the north and south of P with grid indices $(i, j+1)$ and $(i, j-1)$ respectively. Δx and Δy are the grid-spacings in the X and Y directions respectively. They are not equal to each other and neither are they necessarily uniform. If the partial derivatives in an equation are replaced by approximate algebraic difference quotients, where these difference quotients are expressed strictly in terms of the values of the flow-field variables at the discrete grid points only, the original partial differential equation has been discretized. Moreover, this method of discretization is called the *Finite Difference Method*.

2.3 Boundary Conditions

The problem under consideration involves two square-shaped wall-mounted ribs of side a equal to unity, across which a free stream with X -component of velocity equal to $u_\infty = 1$ m/s is blown. With respect to Fig. 2, the dimensions are: $l_u = 5$ m, $s = 2$ m, $l_d = 20$ m and $h = 6$ m. Thus, the domain has the rectangular dimensions 29 m $\times$ 6 m. For this problem, the boundary conditions are discussed in terms of ψ and ω .

The boundary conditions on velocity are the no-penetration boundary condition and the no-slip boundary condition. According to the no-penetration boundary condition, at the fluid and wall interface, the normal component of the velocity of the fluid must be equal to the normal component of the local wall velocity. According to the no-slip condition the tangential component of the fluid at the interface must be equal to the tangential component of the local wall velocity. For a stationary wall, both the local tangential and normal components are zero which results in $u = 0$ and $v = 0$ at the interfaces.

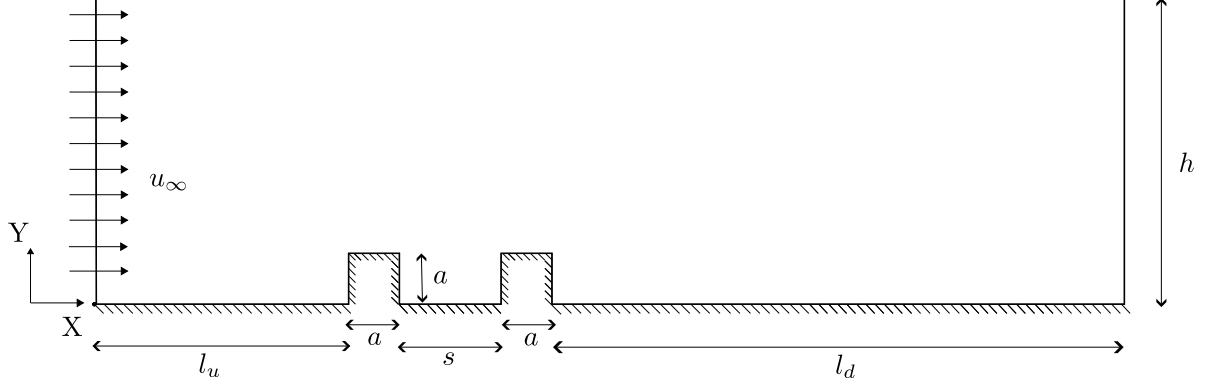


Figure 2: A schematic of the domain, showing the two surface-mounted ribs, the inlet free stream velocity u_∞ and the parameters l_u , a , s , h and l_d .

At $x = 0$ and $0 < y < h$, we have $u = u_\infty$ and $v = 0$, which implies:

$$\frac{\partial \psi}{\partial y} = u_\infty, \quad \frac{\partial \psi}{\partial x} = 0$$

which means that along $x = 0$, ψ depends only on y and we get the boundary condition at the left boundary or inlet as:

$$\begin{aligned} \int_{\psi_0}^{\psi} d\psi &= \int_0^y u_\infty dy \\ \psi &= \psi_0 + u_\infty y \end{aligned} \tag{2.18}$$

For the top boundary we again have $u = u_\infty$ and $v = 0$ giving us the same values for the partial derivatives of ψ as in the case of the inlet. This results in:

$$\psi = \psi_0 + u_\infty h \tag{2.19}$$

For the right boundary or outlet, we have the convective boundary condition:

$$\frac{\partial \psi}{\partial x} = 0 \tag{2.20}$$

The no-penetration and no-slip conditions at each of the wall interfaces of the blocks and the bottom plane results in ψ assuming a constant value at these surfaces as:

$$\psi = \psi_0 \tag{2.21}$$

The value of the constant ψ_0 is arbitrary and without loss of generality, it is assumed that $\psi_0 = 0$ in the following approach.

For the inlet and the top boundary, the flow is similar to a free stream, so the condition for vorticity is

$$\omega = 0 \tag{2.22}$$

At the outlet, we have the convective condition

$$\frac{\partial \omega}{\partial x} = 0 \tag{2.23}$$

The vorticity boundary condition at the walls is less straightforward, since ω is not directly prescribed by the physical no-slip condition. It must be derived from the local behaviour of ψ

near the wall. For the bottom plane in regions $0 < x < l_u$, $l_u + a < x < l_u + a + s$ and $l_u + 2a + s < x < l_u + 2a + s + l_d$, let us take the value of the stream function at a point P on the plane as ψ_P . On this boundary, the value of the stream function is the constant ψ_0 . Due to the no-slip and no penetration boundary conditions, we have:

$$\frac{\partial \psi}{\partial x} = 0, \quad \frac{\partial \psi}{\partial y} = 0$$

Finite difference representations of derivatives are mostly based on Taylor's series expansions. The value of the stream function at the point N which is immediately north of P can be expressed in terms of a Taylor series expanded about point P as follows:

$$\psi_N = \psi_P + \left(\frac{\partial \psi}{\partial y} \right)_P (y_N - y_P) + \left(\frac{\partial^2 \psi}{\partial y^2} \right)_P \frac{(y_N - y_P)^2}{2!} + \text{higher order terms} \quad (2.24)$$

Now, as the point P is on the lowermost plane of the domain, there is no neighbouring point to the south. But for this analysis, we shall assume a point S to its immediate south with the same grid spacing between P and S as that between P and N. Thus, S can be thought of as the mirror image of point N with respect to P. The value of the stream function at S is expressed as:

$$\psi_S = \psi_P + \left(\frac{\partial \psi}{\partial y} \right)_P (y_S - y_P) + \left(\frac{\partial^2 \psi}{\partial y^2} \right)_P \frac{(y_S - y_P)^2}{2!} + \text{higher order terms} \quad (2.25)$$

Now, eliminating the second order-partial derivative and ignoring the higher order terms from equations (2.24) and (2.25), we get:

$$\left(\frac{\partial \psi}{\partial y} \right)_P = \frac{y_P - y_S}{(y_N - y_P)(y_N - y_S)} \psi_N - \frac{y_N - y_P}{(y_P - y_S)(y_N - y_S)} \psi_S + \frac{y_N + y_S - 2y_P}{(y_N - y_P)(y_P - y_S)} \psi_P \quad (2.26)$$

Substituting $(y_N - y_P) = (y_P - y_S)$ and the value of the derivative as zero,

$$\psi_S = \psi_N \quad (2.27)$$

Now, as ψ is constant along X and its partial derivative along that direction is also zero, we have

$$\omega_P = \left(\frac{\partial^2 \psi}{\partial y^2} \right)_P \quad (2.28)$$

If we eliminate $\left(\frac{\partial \psi}{\partial y} \right)_P$ from equations (2.24) and (2.25) and using Eq. (2.27),

$$\omega_P = \frac{2(\psi_P - \psi_N)}{(y_N - y_P)^2} \implies \omega_P = \frac{2(\psi_0 - \psi_N)}{(y_N - y_P)^2} \quad (2.29)$$

This is the boundary condition for horizontal walls with fluid on top such as the bottom plane regions mentioned above as well as the top horizontal surface of the ribs. Similarly, for vertical walls, we do the same analysis for a point P and assume the mirror image of the point immediately to the east or west of P. For vertical walls with fluid on the left, we have:

$$\omega_P = \frac{2(\psi_0 - \psi_W)}{(x_P - x_W)^2} \quad (2.30)$$

and for vertical walls with fluid on the right, we have:

$$\omega_P = \frac{2(\psi_0 - \psi_E)}{(x_E - x_P)^2} \quad (2.31)$$

2.4 Discretization of the Poisson's equation

If we eliminate $\left(\frac{\partial\psi}{\partial y}\right)_P$ from equations (2.24) and (2.25) and ignore the higher order terms, we obtain:

$$\left(\frac{\partial^2\psi}{\partial y^2}\right)_P = \frac{2}{(y_N - y_S)} \left[\frac{\psi_N - \psi_P}{y_N - y_P} + \frac{\psi_S - \psi_P}{y_P - y_S} \right] \quad (2.32)$$

Similarly, we can express ψ_E and ψ_W in terms of the Taylor series expansion about point P and obtain an expression for the second order partial derivative of ψ with respect to x

$$\left(\frac{\partial^2\psi}{\partial x^2}\right)_P = \frac{2}{(x_E - x_W)} \left[\frac{\psi_E - \psi_P}{x_E - x_P} + \frac{\psi_W - \psi_P}{x_P - x_W} \right] \quad (2.33)$$

Now, substituting Eq. (2.33) and Eq. (2.32) in the Poisson's equation Eq. (2.17) and multiplying both sides of the equation by $(x_E - x_W)(y_N - y_S)$ we get:

$$\begin{aligned} & - \left[\frac{(y_N - y_S)(x_E - x_W)}{(x_E - x_P)(x_P - x_W)} + \frac{(y_N - y_S)(x_E - x_W)}{(y_N - y_P)(y_P - y_S)} \right] \psi_P + \frac{y_N - y_S}{x_E - x_P} \psi_E \\ & + \frac{y_N - y_S}{x_P - x_W} \psi_W + \frac{x_E - x_W}{y_N - y_P} \psi_N + \frac{x_E - x_W}{y_P - y_S} \psi_S = -\frac{\omega_P}{2} (x_E - x_W)(y_N - y_S) \end{aligned} \quad (2.34)$$

which can be written as:

$$a_S \psi_S + a_W \psi_W + a_P \psi_P + a_E \psi_E + a_N \psi_N = b_P \quad (2.35)$$

where the coefficients are:

$$a_S = \frac{x_E - x_W}{y_P - y_S}, \quad a_W = \frac{y_N - y_S}{x_P - x_W}, \quad a_P = - \left[\frac{(y_N - y_S)(x_E - x_W)}{(x_E - x_P)(x_P - x_W)} + \frac{(y_N - y_S)(x_E - x_W)}{(y_N - y_P)(y_P - y_S)} \right] \quad (2.36)$$

$$a_E = \frac{y_N - y_S}{x_E - x_P}, \quad a_N = \frac{x_E - x_W}{y_N - y_P}, \quad b_P = -\frac{\omega_P}{2} (x_E - x_W)(y_N - y_S)$$

For each point in the grid, there is a corresponding equation Eq. (2.35) which involves the values of the stream function at that point and the points towards its east, west, north and south. This results in a system of linear equations where the number of equations is equal to the number of grid points. The coefficient matrix A for the system contains values only along five diagonals corresponding to a_S, a_W, a_P, a_E and a_N while the rest of it is occupied by zeroes. This occurs as every point is coupled only with its immediate four neighbours. Such a matrix is a sparse matrix. The way the values are filled along the five diagonals corresponds to the non-zero pattern of A or $NZ(A)$.

$$A = \begin{bmatrix} \ddots & & & & \\ \ddots & \ddots & & & \\ & \ddots & \ddots & & \\ & & a_W & a_P & a_E & \\ \ddots & & & \ddots & \ddots & \ddots \\ & a_S & & \ddots & \ddots & \ddots \\ & & \ddots & & \ddots & \ddots \end{bmatrix}$$

For an irrotational flow, we have $b_P = 0$ for all the equations in the system. This converts the Poisson's equation into a Laplace's equation in terms of ψ

$$a_S \psi_S + a_W \psi_W + a_P \psi_P + a_E \psi_E + a_N \psi_N = 0 \quad (2.37)$$

2.5 Discretization of the vorticity transport equation

The vorticity transport equation (2.15) is a time-dependent equation. For stability reasons, it is more appropriate to choose an implicit approach while discretizing the vorticity transport equation than an explicit one. An implicit approach will result in a system of linear equations. The scheme for the first and second order partial derivatives given in equations (2.26) and (2.33) is called a central difference scheme as it uses the values at the north and south neighbours as well as the current point. For time marching, we adopt a forward difference scheme which considers the values of ω at the current and the next time step only.

$$\left(\frac{\partial \omega}{\partial t}\right)_P = \frac{\omega_P^{n+1} - \omega_P^n}{\Delta t} + O(\Delta t) \quad (2.38)$$

where $O(\Delta t)$ represents the error with an order of magnitude equal to Δt . Using a forward difference quotient for time and central difference quotients for space we get the **Forward-Time Centered-Space (FTCS)** scheme.

$$\begin{aligned} & \frac{\omega_P^{n+1} - \omega_P^n}{\Delta t} + u_P^n \left[\frac{x_P - x_W}{(x_E - x_P)(x_E - x_W)} \omega_E^{n+1} - \frac{x_E - x_P}{(x_P - x_W)(x_E - x_W)} \omega_W^{n+1} + \frac{x_E + x_W - 2x_P}{(x_E - x_P)(x_P - x_W)} \omega_P^{n+1} \right] \\ & + v_P^n \left[\frac{y_P - y_S}{(y_N - y_P)(y_N - y_S)} \omega_N^{n+1} - \frac{y_N - y_P}{(y_P - y_S)(y_N - y_S)} \omega_S^{n+1} + \frac{y_N + y_S - 2y_P}{(y_N - y_P)(y_P - y_S)} \omega_P^{n+1} \right] = \\ & \frac{2}{Re} \left\{ \frac{1}{(x_E - x_W)} \left[\frac{\omega_E^{n+1} - \omega_P^{n+1}}{x_E - x_P} + \frac{\omega_W^{n+1} - \omega_P^{n+1}}{x_P - x_W} \right] + \frac{1}{(y_N - y_S)} \left[\frac{\omega_N^{n+1} - \omega_P^{n+1}}{y_N - y_P} + \frac{\omega_S^{n+1} - \omega_P^{n+1}}{y_P - y_S} \right] \right\} \end{aligned} \quad (2.39)$$

This can be cast as:

$$a_S \omega_S^{n+1} + a_W \omega_W^{n+1} + a_P \omega_P^{n+1} + a_E \omega_E^{n+1} + a_N \omega_N^{n+1} = b_P \quad (2.40)$$

where the coefficients are:

$$\begin{aligned} a_W &= -\Delta t \left[\frac{u_P^n (x_E - x_P) + 2/Re}{(x_E - x_W)(x_P - x_W)} \right], \quad a_S = -\Delta t \left[\frac{v_P^n (y_N - y_P) + 2/Re}{(y_P - y_S)(y_N - y_S)} \right], \\ a_P &= 1 + \Delta t \left[\frac{x_E + x_W - 2x_P}{(x_E - x_P)(x_P - x_W)} u_P^n + \frac{y_N + y_S - 2y_P}{(y_N - y_P)(y_P - y_S)} v_P^n \right. \\ & \quad \left. + \frac{2}{Re} \left(\frac{1}{(x_P - x_W)(x_E - x_P)} + \frac{1}{(y_P - y_S)(y_N - y_P)} \right) \right], \quad b_P = \omega_P^n \\ a_E &= \Delta t \left[\frac{u_P^n (x_P - x_W) - 2/Re}{(x_E - x_W)(x_E - x_P)} \right], \quad a_N = \Delta t \left[\frac{v_P^n (y_P - y_S) - 2/Re}{(y_N - y_P)(y_N - y_S)} \right], \end{aligned} \quad (2.41)$$

Here, u_P^n and v_P^n are taken instead of u_P^{n+1} and v_P^{n+1} as their values at the $(n+1)^{\text{th}}$ time step would have resulted in a system of non-linear equations. Values of u_P^n and v_P^n are obtained by partial derivatives of ψ using central differences.

3 Grid generation

3.1 Generating functions

In order to determine the coefficients a_S, a_W, a_P, a_E and a_N , the distribution of grid points in X and Y directions has to be established. A grid which does not have a uniform spacing in either direction is called a non-uniform grid. Major variation in the flow occurs near the boundaries or walls. In other words, the derivatives have larger values near the boundaries. In finite difference method, the error in the value of a variable depends on the local spacing between points. This can be seen in Eq. (2.24). If the spacing is reduced near boundaries, the truncation error in these expressions also reduces. Hence, grids generated with decreasing values of grid spacing near boundaries or walls result in more accurate solutions compared to that obtained from a grid with uniform spacing. Two generating functions have been used to determine the distribution of grid points.

$$x_i = L \left[\frac{(\beta + 2\alpha) \left(\frac{\beta+1}{\beta-1} \right)^{\left(\frac{\xi_i - \alpha}{1-\alpha} \right)} - \beta + 2\alpha}{(1 + 2\alpha) \left(1 + \left(\frac{\beta+1}{\beta-1} \right)^{\left(\frac{\xi_i - \alpha}{1-\alpha} \right)} \right)} \right] \quad (3.1)$$

Eq. (3.1) generates grid points from 0 to L . Here, $\xi_i = i/(N - 1)$, where N is the number of grid points in the interval $[0, L]$. If we take $\alpha = 0$ and β slightly greater than 1, then Eq. (3.1), generates a grid with points compressed towards L in the interval. Similarly, if we take $\alpha = 0.5$ and β slightly greater than one, we obtain a grid with points compressed towards both boundaries i.e towards zero and L .

$$x_i = L \left[\frac{(\beta + 1) - (\beta - 1) \left(\frac{\beta+1}{\beta-1} \right)^{\left(\frac{\xi_i - \alpha}{1-\alpha} \right)}}{1 + \left(\frac{\beta+1}{\beta-1} \right)^{\left(\frac{\xi_i - \alpha}{1-\alpha} \right)}} \right] \quad (3.2)$$

For values of β slightly greater than 1, Eq. (3.2) generates a grid with points compressed towards zero in the interval $[0, L]$. Moreover, for large values of β , both equations (3.1) and (3.2), generate a uniform grid.

3.2 Division of the domain

The domain is divided into blocks to decide the distribution of grid points in X and Y directions. For the X grid spacing, there are five blocks. The endpoints of the blocks are: $x = 0, x = l_u, x = l_u + a, x = l_u + a + s, x = l_u + 2a + s$ and $x = l_u + 2a + s + l_d$.

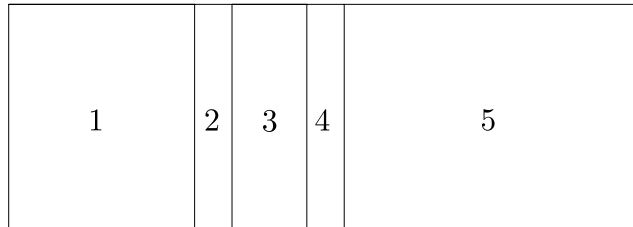


Figure 3: Division of the domain into five blocks along the X direction to control the local grid spacing near the ribs and domain boundaries.

In the first block, the grid becomes compressed towards the right boundary. The second block contains the first rib and is given uniform spacing. The third block has grid points compressed towards both of its boundaries. The fourth block also has uniform spacing as it contains the second rib. Finally, the fifth block has grid points compressed towards the left boundary.

Similarly, we divide the domain into two blocks to decide the spacing in Y direction. The endpoints are: $y = 0, y = a$ and $y = h$. The first block has grid points compressed towards both of its boundaries while the second has points compressed near the lower boundary. The grid size in terms of the number of nodes is 401×101 .

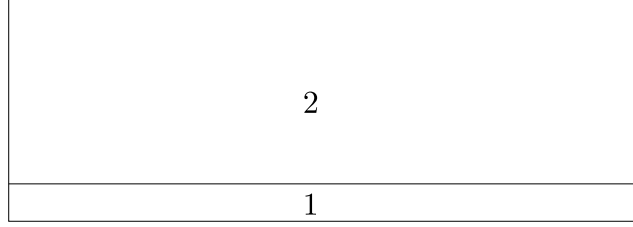


Figure 4: Division of the domain into two blocks along the Y direction to control the local grid spacing near the ribs and bottom wall.

When there is a transition from one block to another, if the change in the distribution of grid points is sudden, it can degrade the quality of the final solution as it affects the gradients at those boundaries. Hence, the transition should be as smooth as possible. Taking the case of grid spacing along X direction, if (i, j) are the indices of a point at a block interface, we define Δx_1 as the difference between the X co-ordinates of the point at (i, j) and $(i - 1, j)$. Similarly, Δx_2 is defined as the difference between the X co-ordinates of the point at $(i + 1, j)$ and (i, j) .

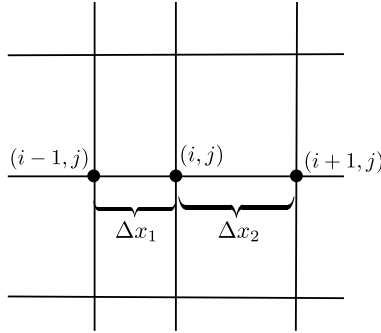


Figure 5: Grid spacings Δx_1 and Δx_2 on either side of a block interface, which are used to define the percentage error ϵ .

We define the percentage error ϵ as follows:

$$\epsilon = \frac{|\Delta x_1 - \Delta x_2|}{\Delta x_1} \times 100$$

At each boundary, the tolerance for ϵ is 0.5%. The values of β were optimised until the ϵ values at block boundaries were less than or equal to the tolerance. Similar optimisation was carried out for blocks for the vertical spacing. In this manner, the grid was generated for the domain. Once the grid points are known, the coefficient matrix can be defined.

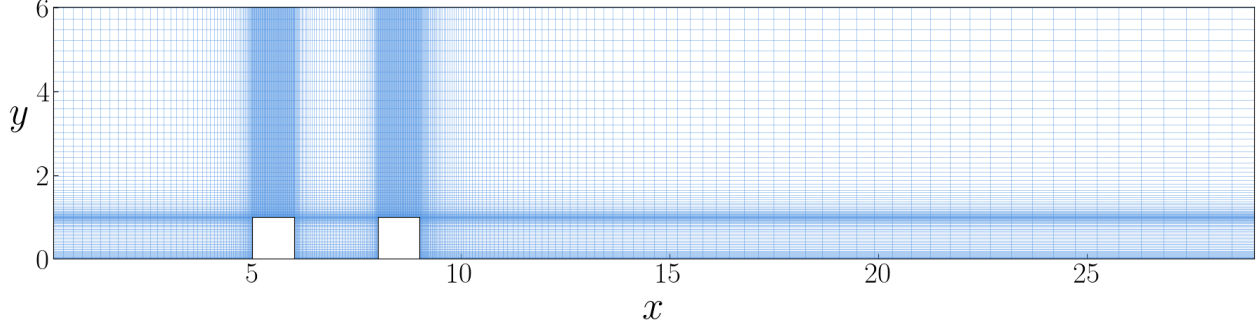


Figure 6: The non-uniform computational grid of the full domain with clustering of grid lines near rib walls and the bottom wall.

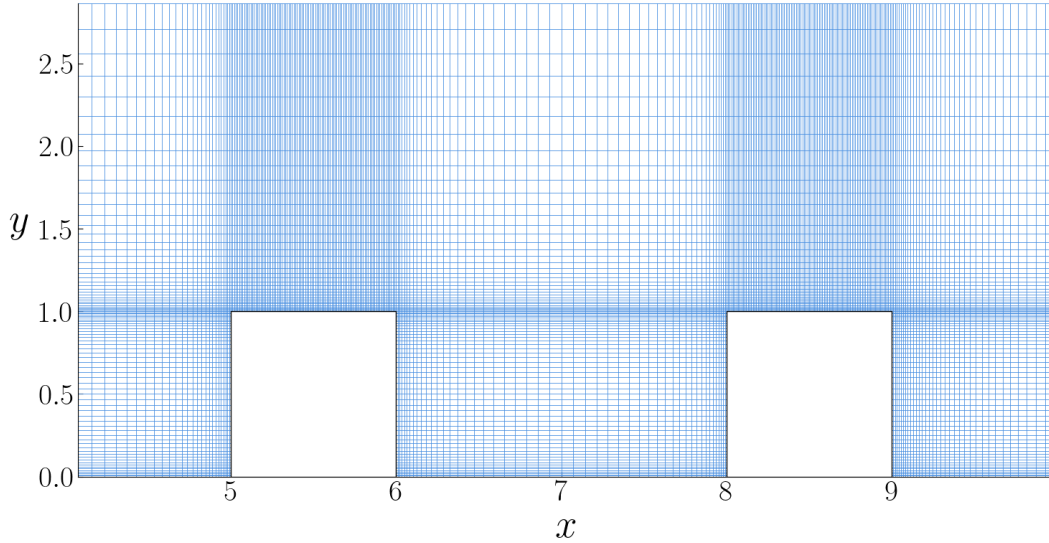


Figure 7: A close-up view of the grid near the two ribs, showing the refinement near the corners and wall boundaries.

4 Methods to solve system of linear equations

As discussed previously, the outcome of discretizing a PDE is a system of linear equations which can be written in the matrix form $Ax = b$, where A contains the coefficients of the variables, x is the vector containing the variables and b is a vector containing the constant terms appearing on the right side of the equations. A system of equations can be solved using direct methods or iterative methods. Among the direct methods, we have Cramer's rule and Gaussian elimination. The algorithm for Cramer's rule is immensely time consuming, with the number of operations being proportional to $(n + 1)!$, where n is the number of unknowns. Cramer's rule should never be used for more than about three unknowns, since it rapidly becomes very inefficient as the number of unknowns increases.

Gaussian elimination is a very useful and efficient tool for solving many systems of algebraic equations, particularly for the special case of a tri-diagonal system of equations. However, the method is not as fast as some others because approximately $n^3/3$ multiplications and divisions are required in solving n equations. Also, round-off errors, which can accumulate through the many algebraic operations, sometimes cause deterioration of accuracy when n is large.

This leaves us with iterative methods. In such methods, an initial guess is made for the values of the variables. Then, each successive iteration updates these values until a point is reached where the solution converges to a particular tolerance.

4.1 Gauss–Seidel and SOR

For a $N \times N$ matrix A , equation $Ax = b$ can be written in terms of its elements as:

$$\sum_{j=1}^N a_{ij}x_j = b_i \implies a_{ii}x_i + \sum_{j=1}^{i-1} a_{ij}x_j + \sum_{j=i+1}^N a_{ij}x_j = b_i$$

where a_{ij} represents the element in the i -th row and j -th column of matrix A , x_j represents the element in the j -th row of vector x and b_i represents the element in the i -th row of vector b . Suppose the algorithm has executed l iterations and the next iteration is the $(l+1)$ -th iteration. The variables are updated in sequential manner. That is, if the current variable to be updated is x_i , then all the variables having indices till i have been updated to the $(l+1)$ -th iteration and those with indices greater than i are yet to be updated. This can be written as:

$$\begin{aligned} a_{ii}x_i^{l+1} &= b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{l+1} - \sum_{j=i+1}^N a_{ij}x_j^l \\ \implies a_{ii}x_i^{l+1} &= a_{ii}x_i^l + b_i - \left(\sum_{j=1}^{i-1} a_{ij}x_j^{l+1} + \sum_{j=i}^N a_{ij}x_j^l \right) \\ \implies x_i^{l+1} &= x_i^l + \frac{1}{a_{ii}} \left[b_i - \left(\sum_{j=1}^{i-1} a_{ij}x_j^{l+1} + \sum_{j=i}^N a_{ij}x_j^l \right) \right] \\ \implies x_i^{l+1} &= x_i^l + \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^N a_{ij}x_j^{\text{latest}} \right) \\ \implies x_i^{l+1} &= x_i^l + \frac{1}{a_{ii}} Res^{\text{latest}} \end{aligned} \tag{4.1}$$

here, x_j^{latest} refers to the latest values available. Res^{latest} is equal to $b_i - \sum_{j=1}^N a_{ij}x_j^{\text{latest}}$ and represents the element of the residual vector $b - Ax$. The variables are updated in this manner until $\|b - Ax\| < \delta$ where δ is the specified tolerance. In order to accelerate convergence, we can use an *over-relaxed* update with $\omega > 1$ as:

$$x_i^{l+1} = x_i^l + \frac{\omega}{a_{ii}} Res^{\text{latest}} \tag{4.2}$$

This x_i^{l+1} is the newly adjusted or “better guess” for x_i^l at the $l+1$ iteration level. That is, we expect x_i^{l+1} to be closer to the final solution than the unaltered x_i^{l+1} from the Gauss–Seidel calculation. This is called **Successive Over-relaxation (SOR)** technique. In some problems,

under-relaxation $0 < \omega < 1$ is employed. Under-relaxation appears to be most appropriate when the convergence at a point is taking on an oscillatory pattern and tending to "overshoot" the apparent final solution. The general range is $0 < \omega < 2$ for satisfactory convergence.

Algorithm 1: SOR Method

Input : A , b , initial guess x_0 , relaxation factor ω , tolerance δ

Output: Approximate solution x

```

1 Initialize:  $x = x_0$ ,  $k = 0$ .
2 while  $\|b - Ax\| > \delta$  do
3   Calculate  $Res^{\text{latest}} = b - Ax$ .
4   Update:  $x_i^{k+1} = x_i^k + \frac{\omega}{a_{ii}} Res^{\text{latest}}$ .
5    $k \leftarrow k + 1$ .
6 end
```

For the problem under discussion, the linear equation for ψ has the form Eq. (2.35). For successive iterations, it follows that the updates are made as:

$$\psi_P^{l+1} = \psi_P^l + \frac{\omega}{a_P} [b_P - (a_W \psi_W^{l+1} + a_S \psi_S^{l+1} + a_P \psi_P^l + a_E \psi_E^l + a_N \psi_N^l)] \quad (4.3)$$

4.2 Preconditioners

Consider the system $Ax = b$. If it is left pre-multiplied by a matrix M^{-1} , then effectively we are solving the modified system $M^{-1}Ax = M^{-1}b$. Our objective is to choose M in such a way that the modified system is significantly easier to solve than the original one. On one extreme, if $M = A$ then we have to solve $x = M^{-1}b$. This directly solves the system. However, inverting the matrix $M = A$ is as difficult as solving the original system. On the other hand, if $M = I$ then we actually have to solve the same system which we started out with, $I^{-1}Ax = I^{-1}b$, i.e, $Ax = b$. So, M must be chosen based on two factors:

1. Computation of $M^{-1}Ax$ should be inexpensive.
2. M should be as close as possible to A .

Now, if $M = D$ is chosen where D is the diagonal of A , then M is trivial to invert. Such a choice is the **Jacobi preconditioner**. However, it hardly accelerates convergence and hence is a poor choice for a preconditioner.

If the matrix M is constructed such as: $M = LU$ where L and U are lower and upper triangular factors which when multiplied are expected to produce A . Since L and U contain elements only along certain diagonals, we can represent them as: $[L_S \ \cdots \ L_W \ L_P \ \cdots]$ and $[I \ U_E \ \cdots \ U_N \ \cdots]$ where L_W, L_S, L_P, U_E and U_P represent diagonals containing coefficients. Let us compute the diagonals of M for any vector ϕ .

$$\begin{aligned}
& [L_S \ \cdots \ L_W \ L_P \ \cdots] [I \ U_E \ \cdots \ U_N \ \cdots] \phi = M\phi \\
& \implies [L_S \ \cdots \ L_W \ L_P \ \cdots] [\phi_P + U_E \phi_E + U_N \phi_N] = M\phi
\end{aligned}$$

$$\begin{aligned}
\Rightarrow & (L_S \phi_S + L_S U_E^{j-1} \phi_{SE} + L_S U_N^{j-1} \phi_P) \\
& + (L_W \phi_W + L_W U_E^{i-1} \phi_P + L_W U_N^{i-1} \phi_{NW}) \\
& + (L_P \phi_P + L_P U_E \phi_E + L_P U_N \phi_N) = M\phi.
\end{aligned}$$

where the super-script $j - 1$ denotes the values of coefficients at the point south of ϕ_P and the super-script $i - 1$ denotes the values of coefficients at the point west of ϕ_P . We define ϕ_{SE} as the value at the point which is south of ϕ_E , i.e, having indices $(i + 1, j - 1)$ and ϕ_{NW} as the value at the point west of ϕ_N , i.e, having indices $(i - 1, j + 1)$. Comparing both sides,

$$M_W = L_W, M_S = L_S, M_N = L_P U_N, M_E = L_P U_N \text{ and}$$

$$M_P = L_P + L_W U_E^{i-1} + L_S U_N^{j-1}$$

However, there are additional diagonals that appear now, owing to the multiplication, which were absent in A . In other words, the non-zero structure of A is violated due to LU multiplication.

$$M_{NW} = L_W U_N^{i-1}, M_{SE} = L_S U_E^{j-1}$$

To take care of these elements, we say, let $M = LU = A + N$ where N is a matrix having non-zero elements on the extra diagonals that arise owing to LU multiplication.

$$N_{NW} = M_{NW}, N_{SE} = M_{SE}$$

In **Incomplete LU (ILU) factorization**, the role of additional diagonals is completely ignored. We force the diagonals of M which correspond to A to be equal to those of A .

$$\begin{aligned}
L_W &= M_W = a_W \\
L_S &= M_S = a_S \\
M_P &= L_P + L_W U_E^{i-1} + L_S U_N^{j-1} = a_P \\
\Rightarrow L_P &= a_P - L_W U_E^{i-1} - L_S U_N^{j-1} \\
M_N &= L_P U_N = a_N \Rightarrow U_N = a_N / L_P \\
M_E &= L_P U_E = a_E \Rightarrow U_E = a_E / L_P
\end{aligned} \tag{4.4}$$

According to Stone (1968)¹, matrix N can be assumed to have non-zero entries at all locations corresponding to A . In this case, N not only has additional diagonals that arise owing to LU multiplication, but also has its own diagonals that obey $NZ(A)$, i.e,

$$N\phi = N_W \phi_W + N_S \phi_S + N_P \phi_P + N_N \phi_N + N_E \phi_E + M_{NW} \phi_{NW} + M_{SE} \phi_{SE}$$

Additional diagonals of M which do not correspond to A are always “dumped” into N . In ILU factorization, these additional diagonals are completely ignored. But, Stone proposed to account for their effects through the other diagonals of N . Now, M becomes close to A if

$$N\phi = N_W \phi_W + N_S \phi_S + N_P \phi_P + N_N \phi_N + N_E \phi_E + M_{NW} \phi_{NW} + M_{SE} \phi_{SE} = 0$$

In other words, the effect of additional diagonals is completely nullified by the combination of the other diagonals.

$$N\phi = \underbrace{N_N \phi_N + N_W \phi_W + N_P \phi_P}_{-M_{NW} \phi_{NW}^*} + \underbrace{N_S \phi_S + N_E \phi_E}_{-M_{SE} \phi_{SE}^*} + M_{NW} \phi_{NW} + M_{SE} \phi_{SE} = 0$$

$$\implies M_{NW}(\phi_{NW} - \phi_{NW}^*) + M_{SE}(\phi_{SE} - \phi_{SE}^*) = 0$$

Here, ϕ_{NW}^* and ϕ_{SE}^* are approximate values of ϕ_{NW} and ϕ_{SE} obtained from nearby grid points. The form for ϕ_{NW} is shown below, rest follow in the same way. We consider a uniform grid with grid spacing in both directions as Δ . The derived expressions can be extended for a non-uniform grid as well.

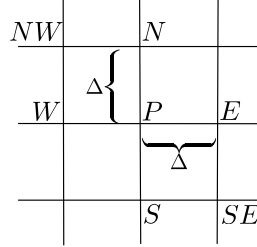


Figure 8: Stencil showing the NW and SE neighbours of point P, which arise from the LU factorization and are treated differently in ILU(0) and SIP preconditioning schemes.

$$\phi = a + bx + cy$$

$$\implies a = \phi_P, \quad a - b\Delta = \phi_W, \quad a + c\Delta = \phi_N$$

$$\implies a = \phi_P, \quad b = (\phi_P - \phi_W)/\Delta, \quad c = (\phi_N - \phi_P)/\Delta$$

$$\implies \phi_{NW} = \phi_N + \phi_W - \phi_P + O(\Delta^2)$$

However, a factor $0 < \alpha < 1$ was introduced to dampen the interpolation and improve convergence.

$$\phi_{NW}^* \approx \alpha(\phi_N + \phi_W - \phi_P), \quad \phi_{SE}^* \approx \alpha(\phi_S + \phi_E - \phi_P)$$

Thus, $N\phi = 0$ becomes:

$$M_{NW}(\phi_{NW} - \alpha(\phi_N + \phi_W - \phi_P)) + M_{SE}(\phi_{SE} - \alpha(\phi_S + \phi_E - \phi_P)) = 0$$

If one compares the above equations with the diagonals of N , we observe that

$$N_S = -\alpha M_{SE}, \quad N_W = -\alpha M_{NW}, \quad N_P = \alpha(M_{NW} + M_{SE})$$

$$N_E = -\alpha M_{SE}, \quad N_N = -\alpha M_{NW}$$

However, since $A + N = M$, the following equations are obtained:

$$\begin{aligned} a_S - \alpha L_S U_E^{j-1} &= L_S, & a_W - \alpha L_W U_N^{i-1} &= L_W \\ a_P + \alpha L_W U_N^{i-1} + \alpha L_S U_E^{j-1} &= L_P + L_W U_E^{i-1} + L_S U \\ a_E - \alpha L_S U_E^{j-1} &= L_P U_E, & a_N - \alpha L_W U_N^{i-1} &= L_P U_N \end{aligned}$$

which can be solved for L and U diagonals

$$\begin{aligned} L_S &= a_S / (1 + \alpha U_E^{j-1}), & L_W &= a_W / (1 + \alpha U_N^{i-1}) \\ L_P &= a_P + \alpha(L_W U_N^{i-1} + L_S U_E^{j-1}) - (L_W U_E^{i-1} + L_S U_N^{j-1}) \\ U_E &= (a_E - \alpha L_S U_E^{j-1}) / L_P, & U_N &= (a_N - \alpha L_W U_N^{i-1}) / L_P \end{aligned} \tag{4.5}$$

This is known as Stone's method or **Strongly Implicit Procedure (SIP)**. It should be noted that for $\alpha = 0$, the above expressions reduce to ILU(0) factors. With $\alpha > 0$, the additional diagonals are considered through the other diagonals of N which makes the splitting more accurate. Thus, SIP is a more accurate method of factorization than conventional ILU. It has been seen from numerical exercises that $\alpha \approx 0.9$ gives the best convergence, though there is no universal value of it which ensures the best convergence for a range of grid structures having different aspect ratios.

On a preconditioned system, one has to perform $M^{-1}Ax$ instead of Ax . Suppose, $Ax = x_1$, then $M^{-1}Ax = M^{-1}x_1 = x_2$ can be computed in two steps. Given x_1 , then $LUx_2 = x_1$.

Step 1: $Ls = x_1$ forward sweep; **Step 2:** $Ux_2 = s$ backward sweep

Thus, with x_1 , one obtains x_2 by two simple steps. That is, M^{-1} is obtained in a very simple and inexpensive way.

4.3 Preconditioned Bi-Conjugate Gradient Stabilized Method

Among many variants of the BiCGSTAB technique, the algorithm proposed by Zhang (1997)² is described here. As this technique does not ensure a monotonic convergence, use of preconditioners not only ensures a smoother convergence, but also accelerates it dramatically. The preconditioner steps are written in red.

Algorithm 2: BiCGSTAB Method

Input : A, b , initial guess ϕ_{in} , tolerance δ

Output: Approximate solution ϕ

1 Initialize: $\phi = \phi_{\text{in}}$, $r = b - A\phi_{\text{in}}$, $\textcolor{red}{Mr}_1 = \textcolor{red}{r}$, $r_1 = r$, $r_0 = r$, $\alpha = \xi = 1$, $k = 0$.

2 **while** $\|b - A\phi\| > \delta$ **do**

3 Compute: $\textcolor{red}{Mp}_1 = \textcolor{red}{Ap}$, $Ap = p_1$, $p = r + \beta(p - \xi Ap)$.

4 Compute: $\alpha = \frac{r \cdot r_0}{Ap \cdot r_0}$.

5 Compute: $z = r - \alpha Ap$.

6 Compute: $\textcolor{red}{Mz}_1 = \textcolor{red}{Az}$, $Az = z_1$, $\xi = \frac{Az \cdot z}{Az \cdot Az}$.

7 Update: $\phi = \phi + \alpha p + \xi z$, $r = z - \xi Az$, $\beta = \frac{\alpha(r_0 \cdot r)}{\xi(r_0 \cdot r)_{k-1}}$.

8 $k \leftarrow k + 1$.

9 **end**

5 Solution of potential flow problem

Before addressing the full viscous flow problem governed by vorticity transport equations, the linear solvers were first implemented to solve the simpler potential flow problem. In the absence of vorticity, the stream function satisfies the Laplace equation as discussed in section 2.1. It serves as a test case where the relative performance of the linear solvers can be evaluated. The system of linear equations obtained by discretizing the Laplace equation was solved using Gauss-Seidel method, SOR method, BiCGSTAB without preconditioner, BiCGSTAB with ILU(0) and BiCGSTAB with SIP. The solvers were allowed to run until the residual value $\|b - A\psi\|_k$ converged to the tolerance, which was taken to be 10^{-8} .

5.1 Convergence behaviour of solvers

Figure 9 shows $\log(r)$ vs $\log(i)$, where r is the residual and i is the number of iterations. On a log plot, the differences between the various methods remain visible throughout the iterations as compared to simply plotting r vs i . Gauss–Seidel method took approximately two hundred and twenty-nine thousand iterations to converge. Using SOR reduced this to around fifty-seven thousand. BiCGSTAB without preconditioning causes a significant reduction with only around six thousand iterations. While ILU(0) took around a hundred and thirty-two iterations and SIP required only sixty-three iterations. Thus, proving the great efficiency of using SIP along with BiCGSTAB for this problem. In comparison, Gauss–Seidel method seems to be a poor choice for solving a system of linear equations as it takes a significantly greater number of iterations. Preconditioners accelerate convergence in BiCGSTAB by decreasing unnecessary fluctuations or smoothing out the declining graph. The absolute CPU time taken by a given solver will depend

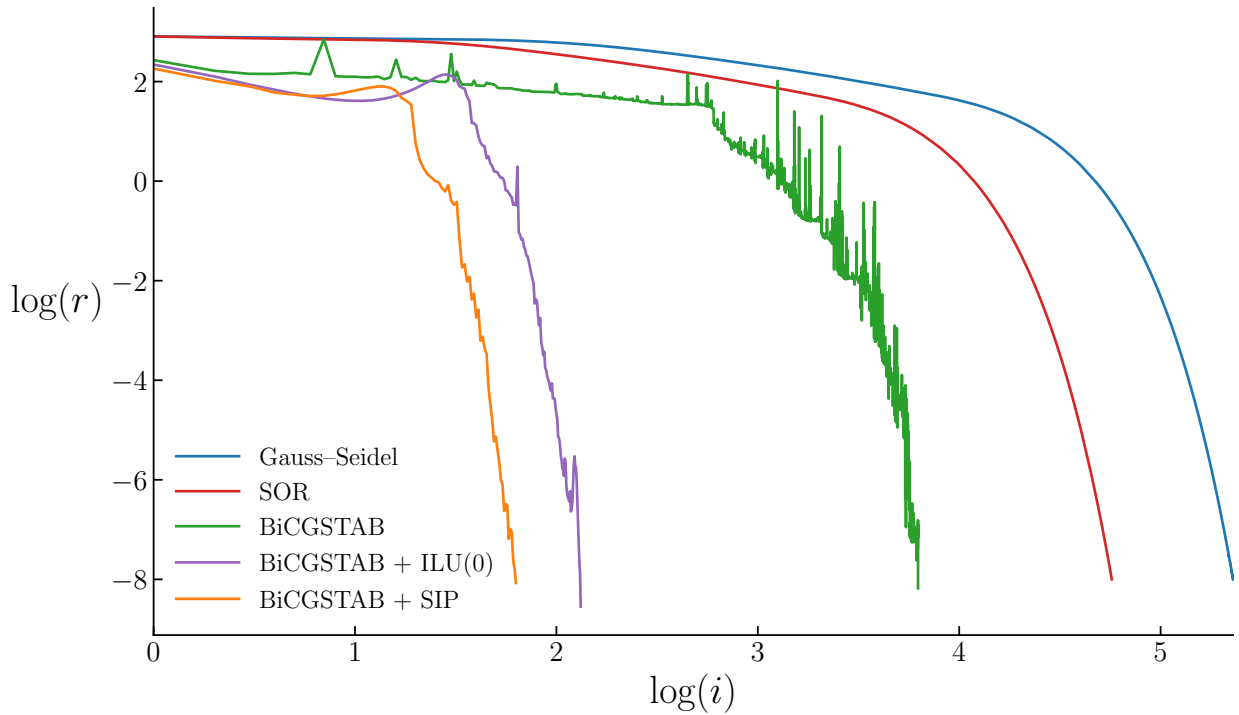


Figure 9: Residual norm $\log(r)$ vs iteration count $\log(i)$ for the five iterative methods applied to the discretized Laplace equation.

on the hardware, compiler optimisations and system load at the time of execution. Hence, the raw timing data is not useful for comparing solver efficiencies across different runs or platforms. To enable a comparison which is independent of the hardware, the CPU time of each solver was normalised with respect to that of BiCGSTAB with SIP preconditioning, which showed the fastest convergence among all the methods tested. The following table shows the CPU time for the execution of the solvers normalised with respect to BiCGSTAB with SIP.

Table 1: CPU time of the iterative methods normalised with respect to BiCGSTAB + SIP.

Method	Normalised CPU time
Gauss–Seidel	982.79
SOR (Gauss–Seidel)	248.73
BiCGSTAB	171.74
BiCGSTAB + ILU(0)	2.12
BiCGSTAB + SIP	1.00

5.2 Flow visualisation

Figure 10 illustrates the streamlines obtained from the numerical solution of the stream function over the two wall-mounted ribs. Away from the obstacles, the streamlines remain almost horizontal and uniformly spaced, which indicates an almost undisturbed free-stream flow. As the fluid approaches the first rib, the flow area reduces. This causes the streamlines to converge and bend upwards. The reduction in flow area must come with an increased fluid velocity in the stream-wise direction according to the continuity equation. Immediately downstream of each rib, the streamlines gradually spread apart as the flow expands back into a larger cross-sectional area. The flow undergoes major change in direction near the leading and trailing edges of the ribs, resulting in large curvature at those locations.

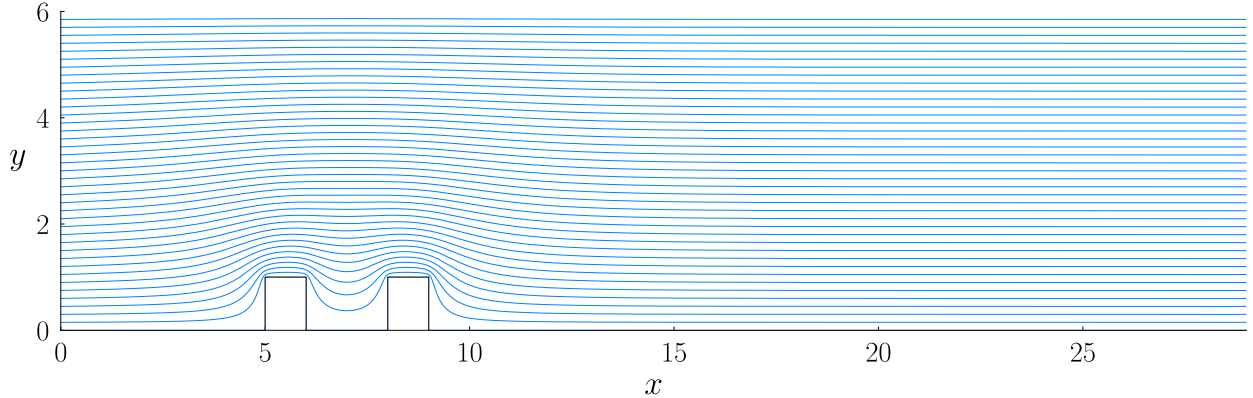


Figure 10: Streamlines over the full computational domain obtained from the potential flow solution, which indicate smooth flow around the ribs without separation.

Since the assumptions for the flow were steady, incompressible and irrotational, no separation or circulation zones are observed behind the ribs. Instead, the streamlines smoothly wrap around the obstacles and merge again downstream. The smooth and continuous nature of the streamlines throughout the entire computational domain indicates that the solution is stable and the boundary conditions have been imposed correctly. The absence of irregularities or discontinuities in streamlines near the ribs indicates that the non-uniform grid captures the flow near sharp corners accurately. Fig. 11 comparatively shows the results obtained by using various methods of solving the system of equations discussed previously. The similarity in the plots indicates that each method was implemented satisfactorily without any errors and successfully converged within the specified tolerance.

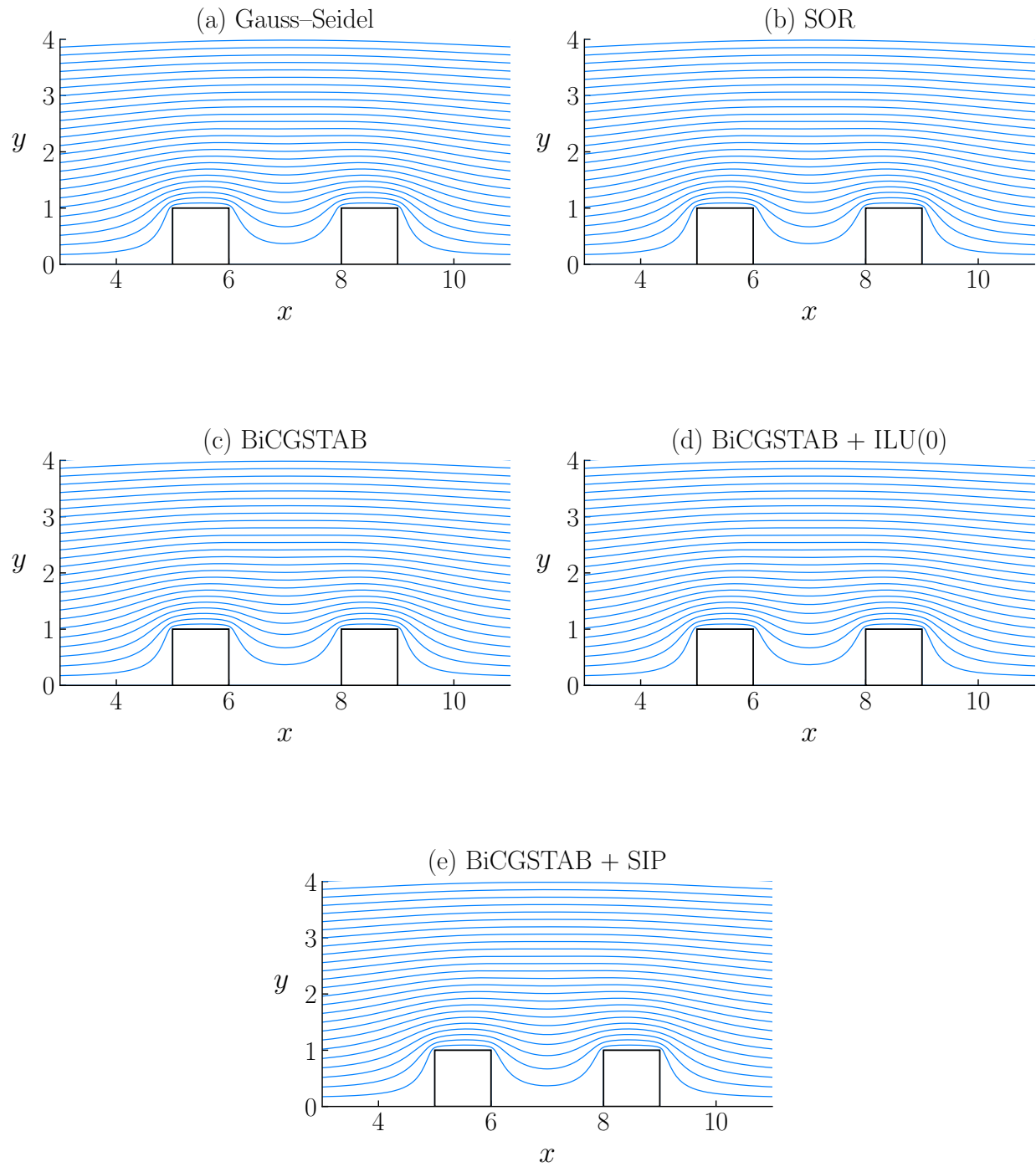


Figure 11: Comparison of the resulting streamlines near the two ribs obtained using each of the five iterative solvers. These results indicate their consistent convergence to the same solution.

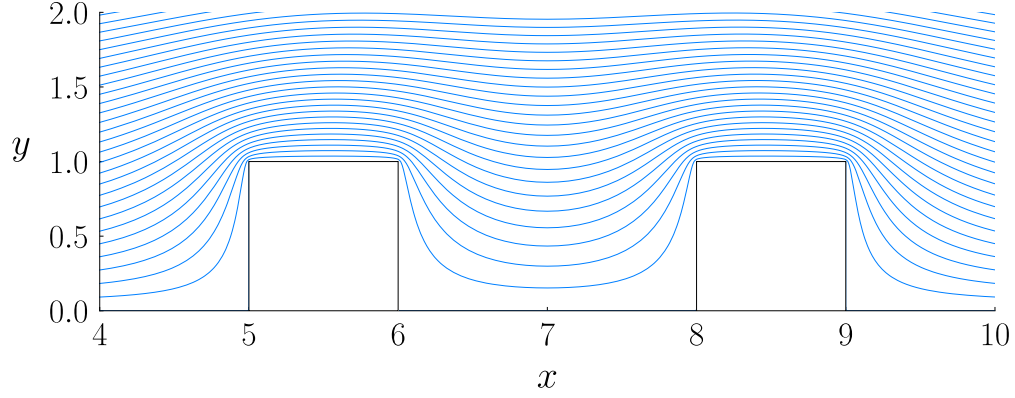


Figure 12: A close-up view of the streamlines near the ribs, which wrap smoothly around the edges and corners.

6 Solution of viscous flow problem

Having obtained the solution for an irrotational flow problem, the analysis is now extended to viscous flow using vorticity-stream function formulation. The effects of fluid viscosity and the associated generation and transportation of vorticity are now taken into consideration.

6.1 Solution procedure

Algorithm 3: Solution Procedure for the Vorticity–Stream Function Formulation

Input : Initial guesses ψ_{in} , ω_{in} , u_{in} , v_{in} , matrices A_ψ , A_ω , vectors b_ψ , b_ω , tolerances δ , ϵ

Output: Steady-state stream function ψ and vorticity ω

- 1 Initialize $k, t = 0$, $\omega_0 = \omega_{in} + K$ where K is a large number;
 - 2 **while** $\text{RMS}(\omega) > \epsilon$ **do**
 - 3 Set: $\omega_0 = \omega$
 - 4 Compute: u and v from ψ
 - 5 Apply velocity boundary conditions
 - 6 Update: coefficient matrix A_ω using u and v
 - 7 Update: $b_\omega = \omega_0$
 - 8 Update: A_ω and b_ω to incorporate vorticity boundary conditions
 - 9 Solve: $A_\omega \omega = b_\omega$ using the BiCGSTAB solver with SIP preconditioning with tolerance δ
 - 10 Compute: b_ψ
 - 11 Update: b_ψ according to the stream function boundary conditions
 - 12 Solve: $A_\psi \psi = b_\psi$ using the BiCGSTAB solver with SIP preconditioning with tolerance δ
 - 13 $t \leftarrow t + \Delta t$
 - 14 $k \leftarrow k + 1$
 - 15 **end**
 - 16 **return** ψ, ω
-

The discretized forms of vorticity transport equation and the Poisson's equation have been discussed in the previous sections. Since the coefficient matrix A_ω for the system of equations involving ω involves velocity components u and v , this system is coupled with the system arising from the discretization of the Poisson's equation. Algorithm 3 illustrates the procedure for solving the two coupled systems of linear equations. For time marching, Δt is chosen by using the Courant condition as a guide and its value is 10^{-3} s in this case. From the previous section which discussed various iterative methods for solving the system of equations obtained from discretizing Laplace equation, it was understood that BiCGSTAB with SIP preconditioning is the most superior. Hence, it has been used to solve both the systems in this problem. The tolerance δ for the solver was taken to be 10^{-6} . The term $\text{RMS}(\omega)$ is defined as:

$$\text{RMS}(\omega) = \sqrt{\frac{(\omega - \omega_0) \cdot (\omega - \omega_0)}{N_x N_y}}$$

where N_x and N_y are number of grid-points along X and Y directions. The tolerance ϵ is taken to be 10^{-6} . The fact that the value of $\text{RMS}(\omega)$ is able to converge to ϵ indicates that the solution reaches a steady-state.

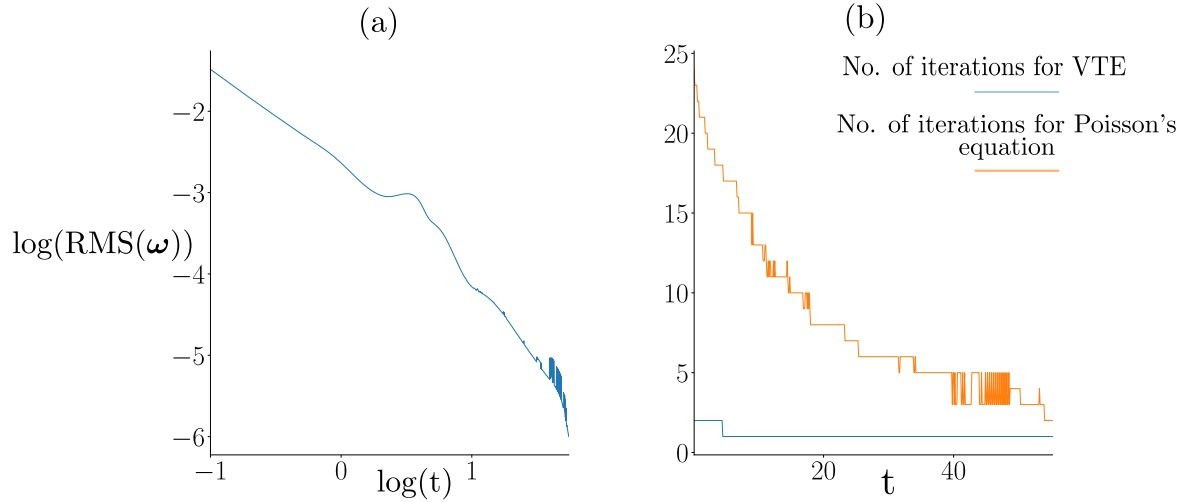


Figure 13: (a) Convergence behaviour of $\log(\text{RMS}(\omega))$ with $\log(t)$ for the viscous flow solution. (b) Number of iterations required by the BiCGSTAB + SIP solver per time step to solve the vorticity transport equation and Poisson's equation.

The solution converges to the specified tolerance at $t = 55.2$. Figure 13 (a) shows that $\text{RMS}(\omega)$ decreases gradually throughout most of the duration but starts fluctuating near the values having an order of 10^{-5} . Figure 13 (b) indicates that the number of iterations required for solving the vorticity transport equation (VTE) is drastically less than the number required for solving the Poisson's equation. This is because the matrix A_ω is diagonally dominant. A matrix A is diagonally dominant if, for every row i :

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Such a matrix reduces the number of iterations required by a solver such as BiCGSTAB by a considerable amount. The number of iterations for VTE starts at two and reaches the constant value of one in a very short duration. In contrast, the number of iterations required for solving the Poisson's equation starts at around twenty-four and decreases in a fluctuating manner to two.

6.2 Flow visualisation

Figure 14 shows the steady-state streamlines over the two ribs at $Re = 100$ and $u_\infty = 1$ m/s. Immediately, we find substantial differences between this case and the one without viscous effects. Multiple recirculation zones are the most notable difference. These regions are characterised by low magnitude of fluid velocity due to dissipation by viscous effects and high magnitude of vorticity arising from shear forces. The large elongated separation zone behind the second block is called the wake. This separated layer eventually curves back and reattaches to the wall further downstream of the second rib. At higher values of Reynolds number, the wake often becomes unsteady and starts shedding vortices periodically. There are three notable regions of recirculation: a small separated bubble upstream of the first rib, the inter-block cavity separation zone and the wake downstream of the second rib.

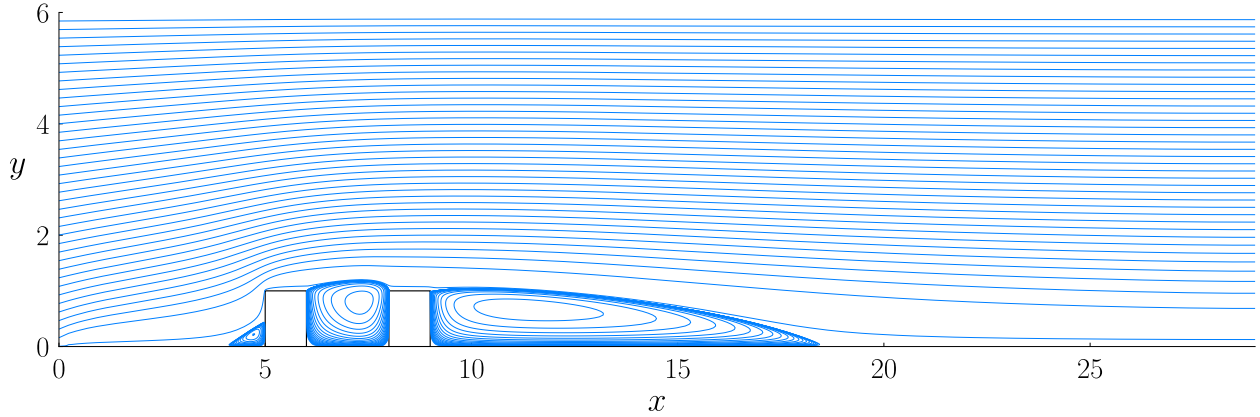


Figure 14: Steady-state streamlines for $Re = 100$ over the computational domain, showing three recirculation zones: the small upstream recirculation bubble, the inter-rib cavity and the downstream wake.

Just ahead of the first rib, near $x \approx 4$ m to $x \approx 5$ m, there is a small recirculation zone hugging the wall. The flow approaching the corner must decelerate and turn at a steep angle. This leads to an adverse pressure gradient near the corner. The flow close to the bottom wall is affected by this adverse pressure gradient and so it separates from the wall just slightly ahead of the first rib. The separated layer rolls up to form this small recirculating zone trapped near the corner. The resolution of this eddy is facilitated by good refinement of the grid near the corners.

A larger recirculation zone fills the cavity between the two blocks from $x = 6$ m to $x = 8$ m. As the flow travels over the first rib, it does not curve downwards into the gap. It mostly skims over the top around the same height as the top of the ribs. However, fluid near the walls of the cavity is moving at near-zero speed due to the no-slip condition and no-penetration condition. Shear forces act between the fast moving top layer and the comparatively slow-moving lower layers, resulting in circulation. This is analogous to the lid-driven cavity problem.

The largest recirculation zone by far is the wake downstream of the second rib. It is a highly elongated region of closely nested streamlines, extending from $x = 9$ m to $x \approx 19$ m. The flow leaving the top surface of the second rib separates due to its inability to turn along the sharp top corner. Circulation occurs due to the fast-moving outer flow and the slow moving region near the trailing edge of the block and the bottom wall surfaces. This continues downstream until the fluid in the outer region is completely decelerated and reattaches to the bottom wall. The reattached flow then resumes moving along the same direction as the main stream.

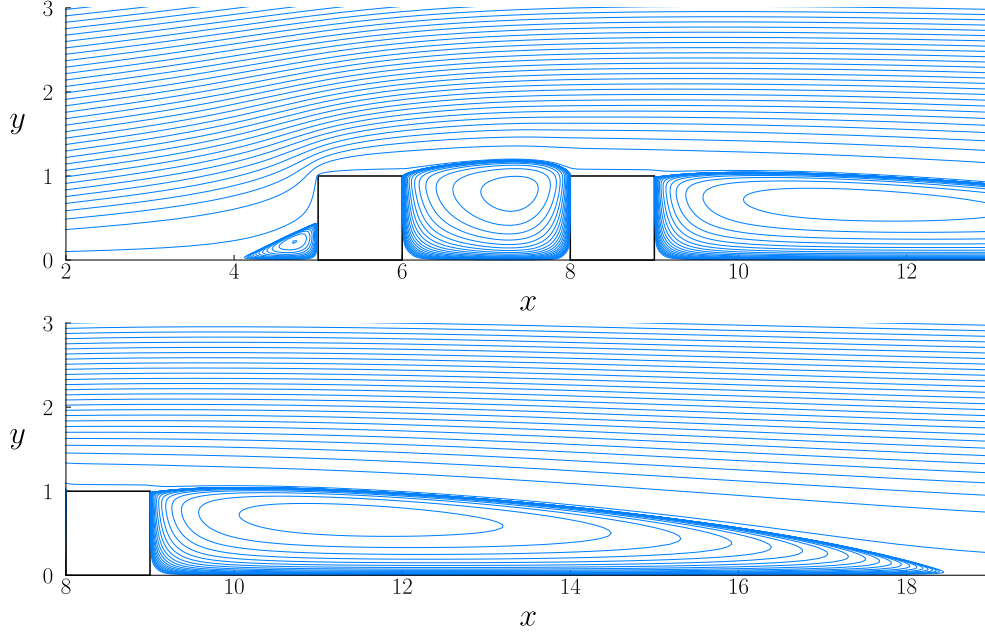


Figure 15: A close-up view of the streamlines near the first rib and cavity (top) and the downstream wake region (bottom), highlighting the recirculating flow.

Figure 15 shows the vorticity contours at steady state. From the inlet to the first rib, the plot shows bands of negative vorticity growing steadily in thickness, indicating a growing viscous boundary layer. At the upstream face and top corner of the first rib, there is an increase in the intensity of negative vorticity. The sharp corner acts as a fixed separation point. Inside the cavity between the ribs, we find alternating bands of positive and negative vorticity, indicating a trapped recirculation zone. Downstream of the second rib, a band of positive vorticity clings to the bottom wall. This marks the reversed near-wall flow at the base of the wake. Further downstream, the contour levels relax and vorticity starts to diffuse gradually. Essentially all the rotational activity in the domain is confined to a thin region near the bottom wall ($y < 2.5\text{-}3\text{ m}$), while the portion above remains mostly free of vorticity.

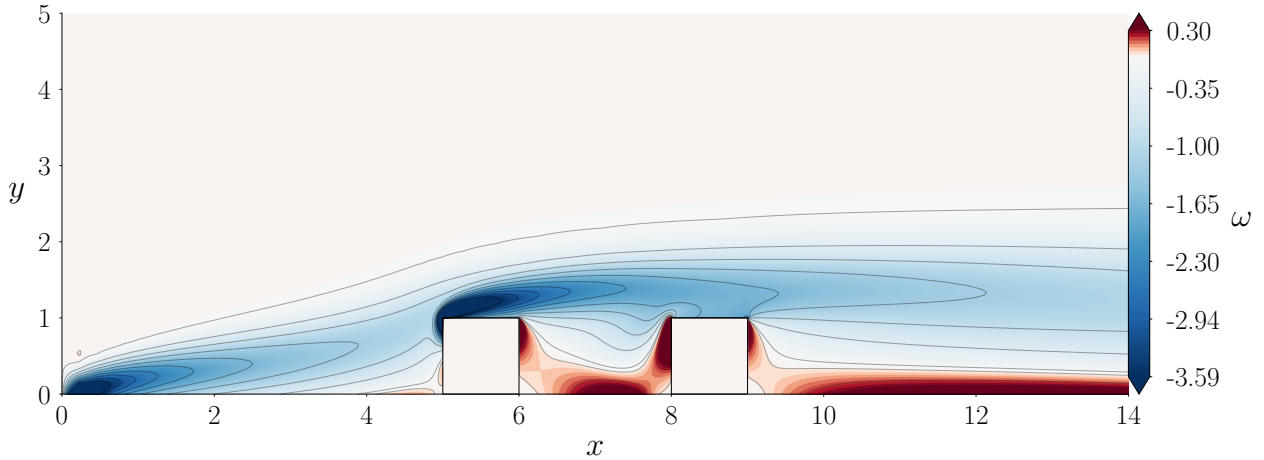


Figure 16: Steady-state vorticity field over the domain, indicating the distinction between regions of attached flow (shown in dark blue) and recirculating flow (shown in red).

7 Conclusion

This report presented a numerical study of the laminar flow over two surface-mounted ribs using vorticity-stream function formulation, facilitated by the Finite Difference Method on a non-uniform grid. The elliptic Poisson's equation and parabolic vorticity transport equation have been discretized using central difference scheme in space and implicit forward time scheme.

Methods to iteratively solve a system of linear equations were discussed and their convergence behaviour was analysed. From the analysis, we conclude that the use of preconditioners greatly increases the computational efficiency. The BiCGSTAB technique with SIP preconditioning proved to be the most superior among four other iterative methods. Consequently, it was used to solve the systems of equations arising in the irrotational flow and viscous flow cases. The irrotational solution produced smooth streamlines which easily wrapped around the two mounted ribs without any separation near sharp corners, consistent with the inviscid approximation. This served as a verification step to check the effectiveness of the iterative method before introducing viscous effects. Viscous effects at $Re = 100$ showed remarkable differences compared to the former. Notably, flow separation and recirculation zones were clearly observed. A small upstream vortex near the first rib, a trapped recirculation zone in the cavity between the ribs and the large downstream wake are characteristics of this solution. The vorticity gradients associated with the flow effectively explain the flow behaviour around the ribs.

The results indicate that the vorticity-stream function formulation, combined with a preconditioned iterative solver and a non-uniform mesh can provide a satisfactory description of complex flow phenomena in the laminar regime.

References

- [1] H. L. Stone, "Iterative solution of implicit approximations of multidimensional partial differential equations," *SIAM Journal on Numerical Analysis*, vol. 5, no. 3, pp. 530–558, 1968.
- [2] S.-L. Zhang, "Gpbi-cg: Generalized product-type methods based on bi-cg for solving nonsymmetric linear systems," *SIAM Journal on Scientific Computing*, vol. 18, no. 2, pp. 537–551, 1997.
- [3] J. Anderson, *Computational Fluid Dynamics*. Computational Fluid Dynamics: The Basics with Applications, McGraw-Hill Education, 1995.
- [4] J. Ferziger and M. Peric, *Computational Methods for Fluid Dynamics*. Mathematics and Statistics, Springer Berlin Heidelberg, 2012.
- [5] P. J. Pritchard and J. W. Mitchell, *Fox and McDonald's introduction to fluid mechanics*. John Wiley & Sons, 2016.
- [6] F. White, *Fluid Mechanics*. McGraw-Hill series in mechanical engineering, McGraw Hill, 2011.